Manufacturing Operational Insights Report

Dockerized Healthcare Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	**Mayuri Patidar**	**EN22CS301593**
02	**Meet Solanki**	**EN22CS301595**
03	**Mokshika Yadav**	**EN22CS301614**
04	**Nirdesh Sharma**	**EN22CS301655**
05	**Palak Shahdadpuri**	**EN22CS301677**
06	**Pankaj**	**EN22CS301679**

Group Name: 11 D6

Project Number : DO-22

Industry Mentor Name: Vaibhav

University Mentor Name: Akshay Saxena

Academic Year: 2025-2026

1. Introduction

The Dockerized Healthcare Service project focuses on developing and deploying a healthcare web service using modern DevOps practices. The application is built using Python and the Flask framework, which provides simple API services for handling basic healthcare-related operations such as patient information management and service responses.

To solve common deployment and dependency issues, the application is containerized using Docker. Docker packages the application along with all required libraries and configurations into a container, ensuring that the system runs consistently across different environments without compatibility problems. This makes the application easy to deploy, portable, and reliable.

By using containerization, the healthcare service can be quickly deployed on any system that supports Docker. It also allows the system to scale easily by running multiple container instances when needed. Overall, this project demonstrates how Docker-based deployment improves application portability, consistency, and efficiency in modern web service development.

1.1 Scope of the document

This document describes the Low-Level Design (LLD) of the Dockerized Healthcare Service. It explains the internal structure and detailed implementation of the system, including how different components interact with each other. The document covers the design of the healthcare web service, the containerization process using Docker, and the deployment workflow used to run the application.

It also outlines the technologies used in the project, such as the Flask framework for building the healthcare API and Docker for packaging the application with its dependencies. In addition, the document explains how the system ensures environment consistency, portability, and reliable deployment through container-based architecture.

The purpose of this document is to provide a clear understanding of how the healthcare service is designed, developed, and deployed using modern DevOps practices. It serves as a reference for developers, testers, and project reviewers to understand the system implementation and deployment workflow.

1.2 System overview

The Dockerized Healthcare Service is a web-based system designed to provide basic healthcare-related services through a simple and efficient API interface. The main goal of the system is to support healthcare operations such as managing patient-related information and responding to healthcare service requests. The application is designed in a way that allows users or systems to interact with the service and receive appropriate healthcare-related responses.

To ensure smooth deployment and reliable execution across different environments, the system uses Docker containerization technology. Containerization allows the entire application, along with its required dependencies and configurations, to be packaged into a single container. This ensures that the application runs consistently regardless of the operating system or environment in which it is deployed. By using containers, the system avoids common issues related to dependency conflicts and manual environment setup.

The Dockerized architecture also improves the portability and scalability of the healthcare service. Since the application is packaged inside a container, it can be easily moved and deployed on different machines, servers, or cloud environments that support Docker. This approach makes the system more flexible and easier to manage. If the service needs to handle more requests in the future, additional container instances can be deployed to scale the system efficiently.

The system is designed with the objective of demonstrating how modern deployment techniques and DevOps practices can simplify the management of web services. By using container-based deployment, the healthcare service becomes easier to maintain, more reliable, and capable of running consistently across multiple environments. This approach provides a strong foundation for building scalable and easily deployable healthcare applications in real-world scenarios.

1.3 System Design

The system design of the Dockerized Healthcare Service follows a modern DevOps-based architecture that integrates version control, continuous integration, containerization, and container orchestration technologies. The main objective of this design is to ensure that the healthcare service can be developed, built, deployed, and managed efficiently across different environments.

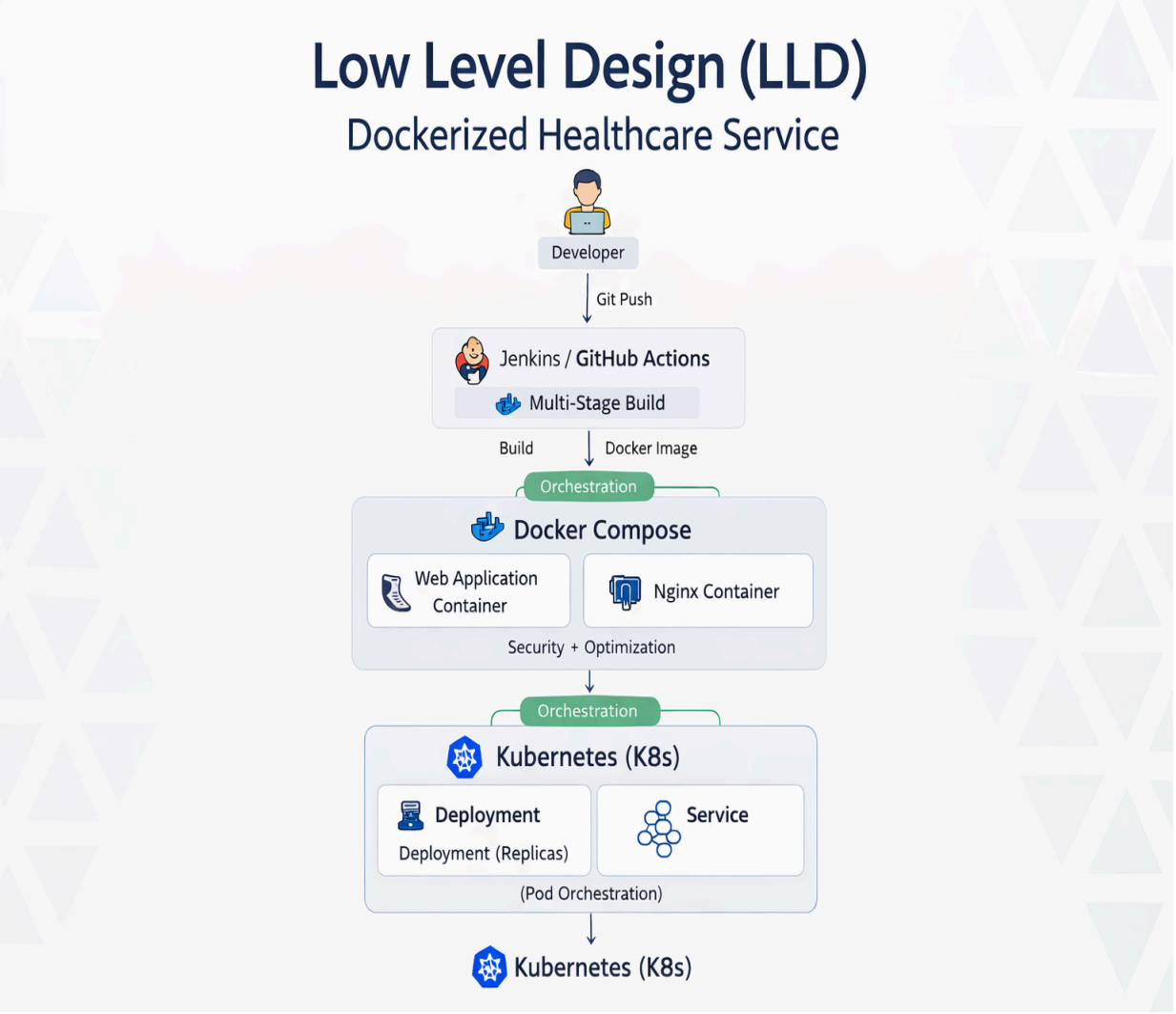
The process begins with the Developer, who writes and updates the application code. Once the code changes are completed, the developer pushes the code to the source repository using Git. This action automatically triggers the CI/CD pipeline managed by tools such as Jenkins or GitHub Actions. These tools automate the process of building the application and preparing it for deployment.

During the CI/CD pipeline stage, a multi-stage build process is executed. In this stage, the system builds a Docker image that contains the complete application along with all necessary dependencies and configurations. This Docker image ensures that the application runs consistently in any environment without dependency conflicts.

After the Docker image is created, the system uses Docker Compose to manage and run multiple containers required for the application. In this architecture, the application runs inside a Web Application Container, while an Nginx Container acts as a reverse proxy to manage incoming requests and improve security and performance. Docker Compose helps coordinate these containers and ensures that they work together as a single service.

For large-scale deployment and orchestration, the system uses Kubernetes (K8s). Kubernetes manages containerized applications by organizing them into pods and controlling how they are deployed and scaled. The Deployment component ensures that multiple instances (replicas) of the application are running to provide high availability and load balancing. The Service component exposes the application so that users or external systems can access the healthcare service.

Overall, this system design combines CI/CD automation, containerization, and container orchestration to create a reliable and scalable deployment environment. It ensures that the healthcare service can be continuously integrated, easily deployed, and efficiently managed in modern cloud-based infrastructures.



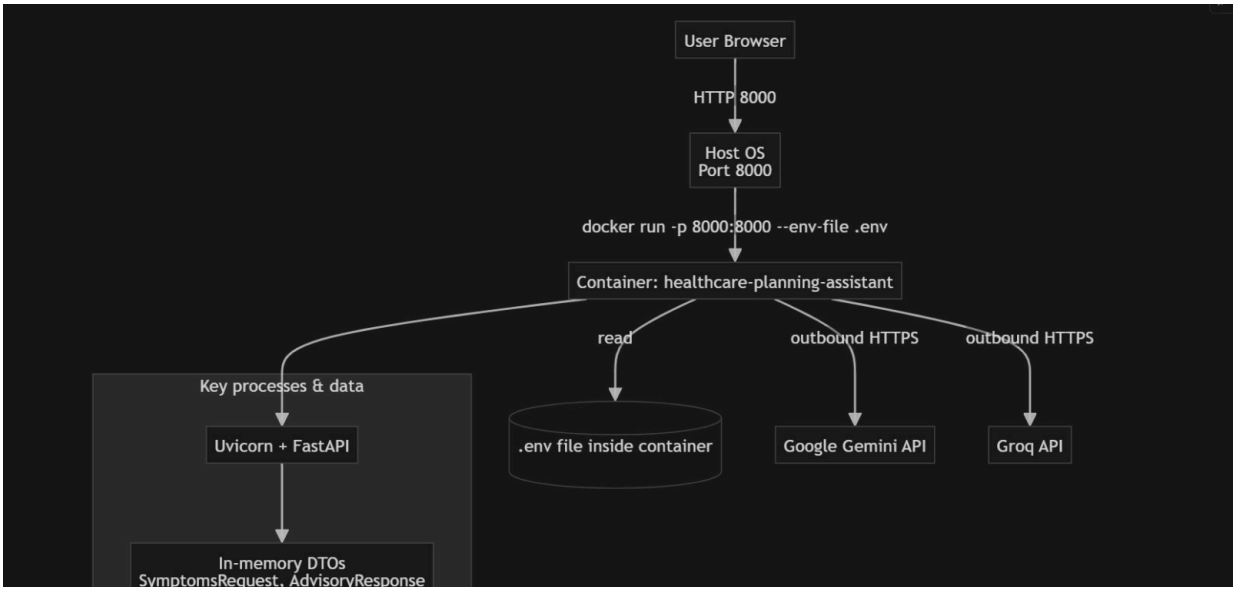
2. Data Design

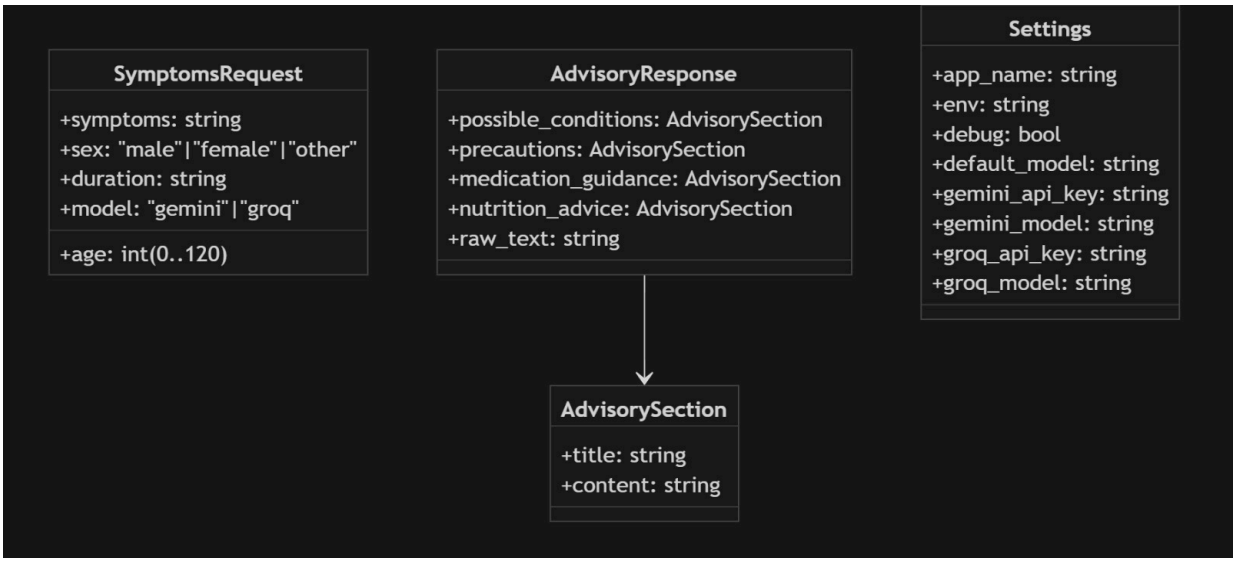
The Data Design of the Dockerized Healthcare Service defines how healthcare-related data is structured and processed within the system. The system handles basic data such as service requests, patient-related information, and response outputs generated by the application.

When a user sends a request to the healthcare service, the system processes the input data and generates the appropriate response. The design ensures that data flows smoothly between different components of the system.

The containerized environment also manages configuration data and environment variables, ensuring that the application runs consistently across different environments. This structured data management helps maintain reliability, consistency, and efficient processing within the healthcare service.

The data design defines the main entities involved in the Dockerized Healthcare Service system and the relationships between them. These entities help manage application deployment, container execution, CI/CD processes, and environment configurations.





3. Interfaces

The Interfaces in the Dockerized Healthcare Service define how different components of the system interact with each other. These interfaces allow communication between users, the application service, and the containerized deployment environment.

The system provides a service interface through which users or external systems can send requests and receive responses from the healthcare service. This interface allows the system to handle healthcare-related operations and return appropriate results.

The system also includes container interfaces that manage communication between different containers running within the Docker environment. These interfaces ensure that the application components can interact smoothly and operate as a single service.

3.1 User Interface:

The User Interface allows users or administrators to interact with the healthcare service system easily. Through the interface, users can access healthcare services, send requests, and view responses from the system. The UI provides a simple and user-friendly way to interact with the deployed application running inside Docker containers.

3.2 Command Line Interface (CLI):

The Command Line Interface is used by developers or system administrators to manage the application and its deployment. Using CLI commands, they can build Docker images, run containers, monitor services, and manage the CI/CD pipeline. It helps automate tasks and control the system efficiently through terminal commands.

4. State and Session Management

State and session management in the Dockerized Healthcare Service ensures that the system can track the status of application components and handle user interactions efficiently during runtime. The system maintains information about running services, container states, and active user requests to ensure smooth operation and service reliability.

In this architecture, container states such as running, stopped, restarted, or failed are continuously monitored to maintain the stability of the deployed healthcare service. If any container fails or stops unexpectedly, the system can detect the issue and restart the container to maintain service availability.

Session management helps in handling user requests and responses during communication with the healthcare service. It ensures that each request is processed correctly and the response is returned to the appropriate user. Proper session handling improves system efficiency, prevents request conflicts, and maintains consistent interaction between users and the deployed services.

Overall, effective state and session management improve system reliability, help monitor containerized services, and ensure smooth communication between users, services, and deployment environments.

5. Functional Requirements

The functional requirements define the specific behaviors and services that the Dockerized Healthcare Service system must provide to support automated deployment, containerization, and management of the healthcare application using modern DevOps tools.

5.1 Application and Repository Management

1. **Source Code Management:**
   The system must allow developers to store and manage the healthcare service source code in a version control repository such as Git.
2. **Version Tracking:**
   The repository must maintain version history of the application code to track changes, updates, and rollbacks if required.
3. **Code Integration:**
   Whenever new code changes are pushed to the repository, the system should automatically trigger the CI/CD pipeline for further processing.

5.2 CI/CD Pipeline Management

1. **Automated Build Process:**
   The system must automatically build the application whenever code changes are committed to the repository.
2. **• Pipeline Automation:**
   The CI/CD pipeline should automate the stages of build, testing, containerization, and deployment using tools such as Jenkins.
3. **• Deployment Trigger:**
   The pipeline must trigger the deployment process after successful completion of the

build stage.

5.3 Docker Containerization

1. **Docker Image Creation:**
   The system must package the healthcare application and its dependencies into a Docker image to ensure consistent execution.
2. **Container Execution:**
   The system must allow Docker containers to run the healthcare application in an isolated environment.
3. **Container Management:**
   Administrators must be able to start, stop, and monitor Docker containers that host the healthcare service.

5.4 Kubernetes Deployment and Orchestration

1. **Container Deployment:**
   The system must deploy containerized healthcare services on Kubernetes clusters.
2. **Service Management:**
   Kubernetes should manage the container lifecycle including scheduling, restarting failed containers, and load balancing.
3. **Scalability:**
   The system should allow scaling of healthcare service containers to handle increased user requests.

5.5 CI/CD Strategy (Continuous Integration and Continuous Deployment)

This section describes the automated pipeline used to deploy the healthcare service.

A. Continuous Integration (CI)

1. **Automated Build:**
   Whenever developers push new code to the repository, the CI pipeline automatically starts the build process.
2. **Code Validation:**
   The system verifies the integrity and correctness of the application code before deployment.
3. **Docker Image Creation:**
   After successful build verification, the pipeline creates a Docker image that contains the complete application.

B. Continuous Deployment (CD)

1. **Container Deployment:**
   The Docker image is deployed as a running container within the container orchestration environment.
2. **Service Availability:**
   The deployment ensures that the healthcare service remains available while updates are being applied.
3. **Automated Updates:**
   Whenever a new version of the application is available, the system automatically updates the running service.

6. Non-Functional Requirements

Non-functional requirements describe the quality attributes and operational standards that the Dockerized Healthcare Service system must maintain. These requirements ensure that the system performs efficiently, securely, and reliably while supporting automated deployment and containerized environments.

6.1 Security Aspects

The system must ensure that all components are protected from unauthorized access and misuse. Access to the source code repository should be restricted to authorized developers only. Sensitive information such as credentials, configuration files, and deployment keys must be securely managed within the deployment pipeline. Containerization also provides isolation between services, which helps prevent interference between different components of the system.

6.2 Performance Aspects

The system should provide efficient deployment and execution of the healthcare service. Automation using CI/CD pipelines and containerization reduces the time required for application deployment and updates. The system should also be capable of handling multiple requests without significant performance degradation. Efficient container management helps maintain smooth and fast application performance.

6.3 Scalability

The system must support scaling of the healthcare service when the workload increases. By using containerized deployment and orchestration tools, multiple container instances can be created to handle higher traffic or additional service requests. This ensures that the application can grow according to demand.

6.4 Reliability and Availability

The system should maintain high availability and reliability during operation. Containers should automatically restart in case of failures, ensuring that the healthcare service remains accessible. Automated deployment and monitoring mechanisms help maintain system stability and reduce downtime.

6.5 Maintainability

The system should be designed in a way that makes it easy to update, manage, and maintain. Using containerization and automated deployment pipelines simplifies system maintenance and allows developers to update the application without affecting the entire infrastructure.

7. References

1. Docker Documentation. (2024). *Docker Official Documentation.* Available at: https://docs.docker.com

2. Jenkins Documentation. (2024). *Jenkins User Documentation.* Available at: https://www.jenkins.io/doc

3. Git Documentation. (2024). *Git Version Control System Documentation.* Available at: https://git-scm.com/docs

4. DevOps Foundation. (2023). *Introduction to DevOps Practices and CI/CD Pipelines.*

5. Kubernetes Documentation. (2024). Kubernetes Official Documentation. Available at: https://kubernetes.io/docs

8. Git Hub: https://github.com/MeetSolanki0/Dockerized-Healthcare-Service

MEDICAPS
UNIVERSITY